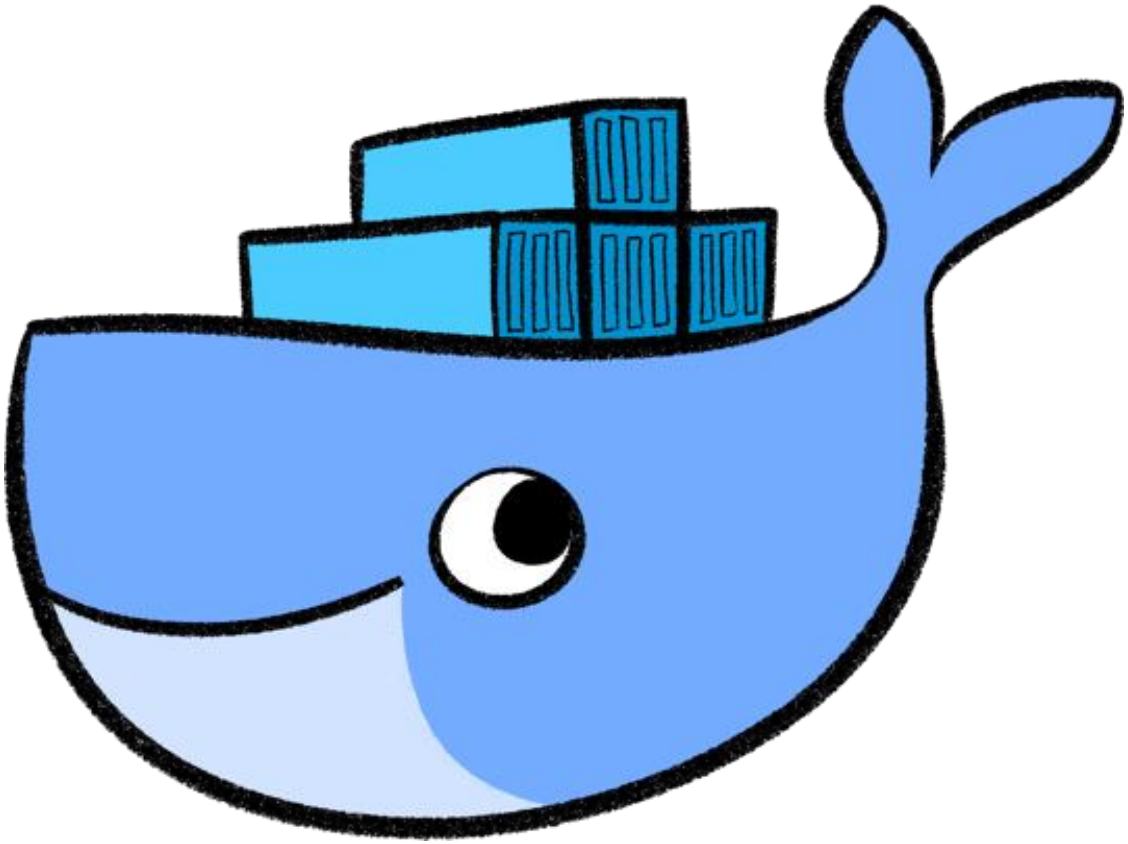
Scaling Dynamic Pages



Slides by Raed Almomani and Andrew DeOrio

Scaling road map

- Last time: scaling static pages
- Today: scaling dynamic pages
- Next time: scaling storage
  - Database
  - Media uploads

Agenda

- Multi-process, multi-thread, and asynchronous
- Round robin DNS and load balancing
- Hardware virtualization
- Containerization

Server-side dynamic pages

- Server-side dynamic pages: Server response is the output of a function.
- What happens when two clients make two requests at the same time?
- What about many clients?

1 server, 1 process, 1 request

- One server runs one process with one thread
  - Example: Flask development server
  - One response at a time

1 server, 1 process, 2 requests

- One server runs one process with one thread

Limitations of 1 process, 1 thread, 1 CPU

- Problem: simultaneous requests go one at a time
- Solution 1: Asynchronous programming
  - Limited solution
  - Illusion of parallelism, still runs on one CPU core
- Solution 1: Multiple threads on one CPU core
  - Limited solution
  - Illusion of parallelism, still runs on one CPU core
- Solution 3: Multiple processes on one CPU core
  - Limited solution
  - Illusion of parallelism, still runs on one CPU core

1 server, 2 processes, 2 requests

- Problem: simultaneous requests go one at a time
- Solution 4: multiple processes or threads on a *multicore CPU*

Multiprocessing with multithreading or asynchronous programming

- Can we combine multiprocessing with multithreading?
  - Yes
  - Example: Apache with mod_wsgi
- Can we combine multiprocessing with asynchronous programming?
  - Yes
  - Example: Gunicorn
- Can we combine multiprocessing with multithreading *and* asynchronous programming?
  - Yes
  - Example: Gunicorn

Many servers

- Problem: one multicore server is not enough
- Solution: buy or rent more servers

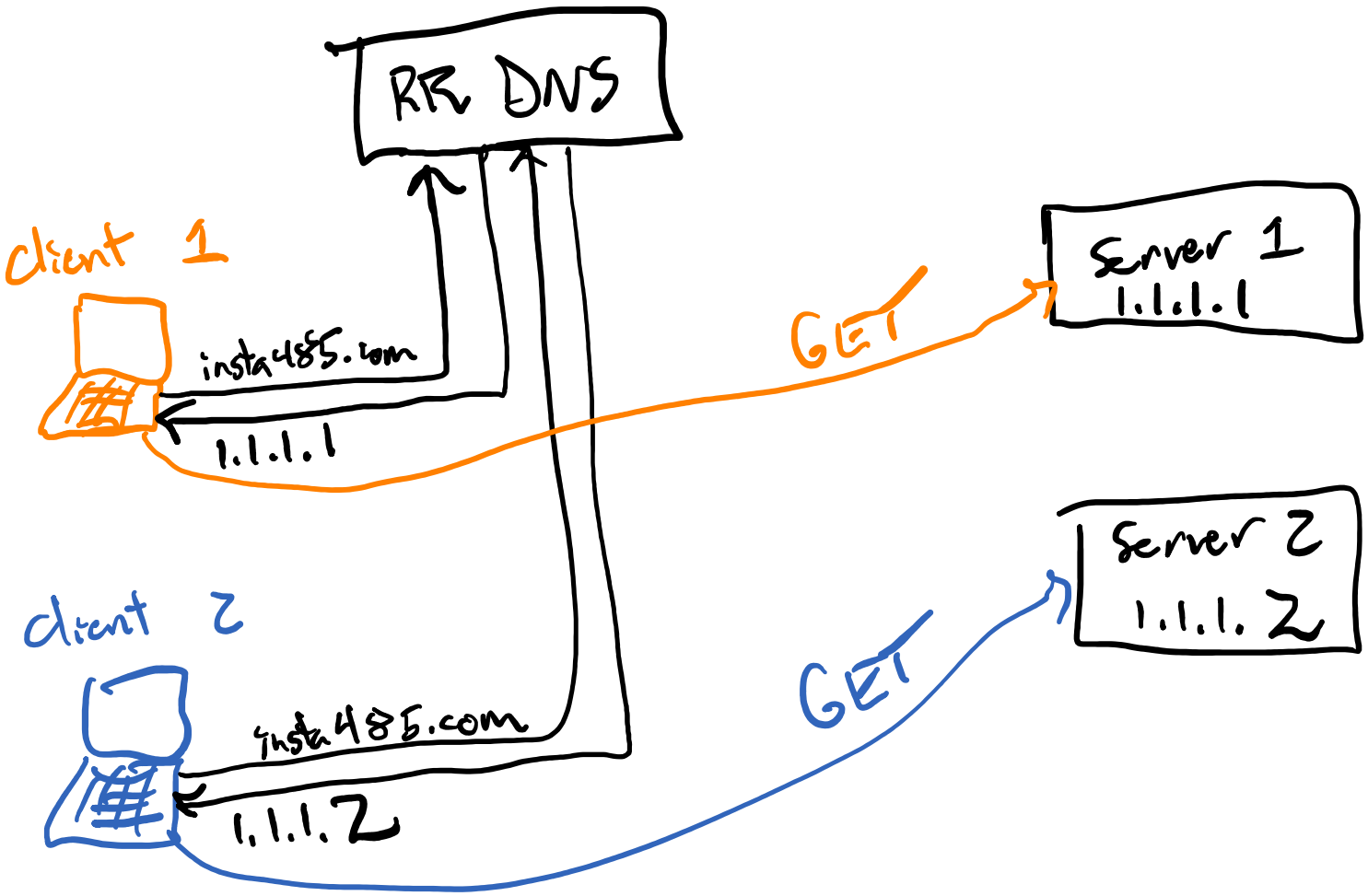
Agenda

- Multi-process, multi-thread, and asynchronous
- **Round robin DNS and load balancing**
- Hardware virtualization
- Containerization

Problems with many servers

- Problem: Two users make a request. Which server should respond?
- Option 1: Round robin DNS

Round Robin DNS diagram



Round Robin DNS description

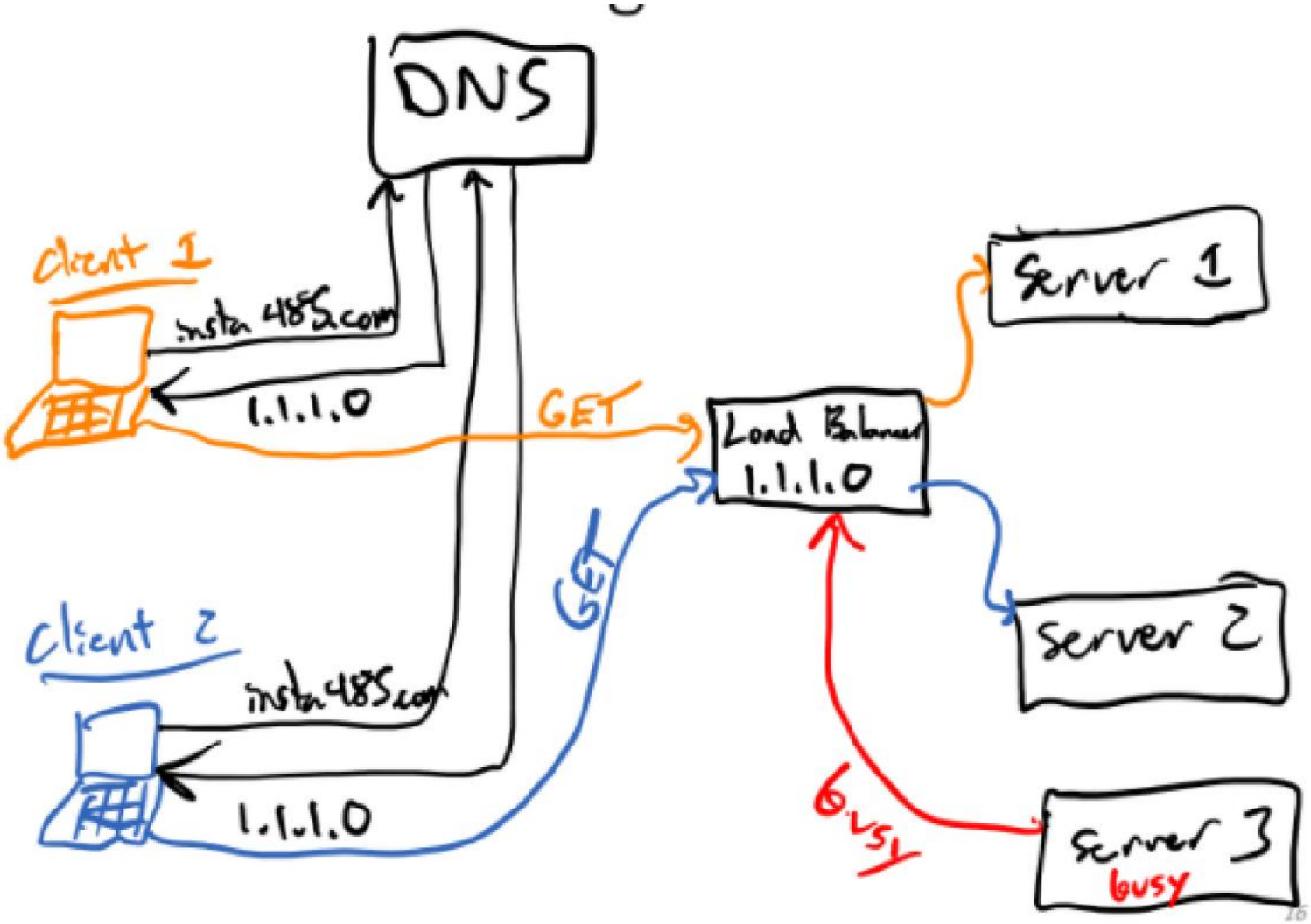
- Modified DNS server responds with different IPs to different dynamic pages web servers
- Rotating list of IPs
- Usually in the same datacenter

Problem with Round Robin DNS

- Problem: round robin DNS doesn’t consider server load. One request might be more time consuming than another

- Solution: Load balancer

Load balancer diagram



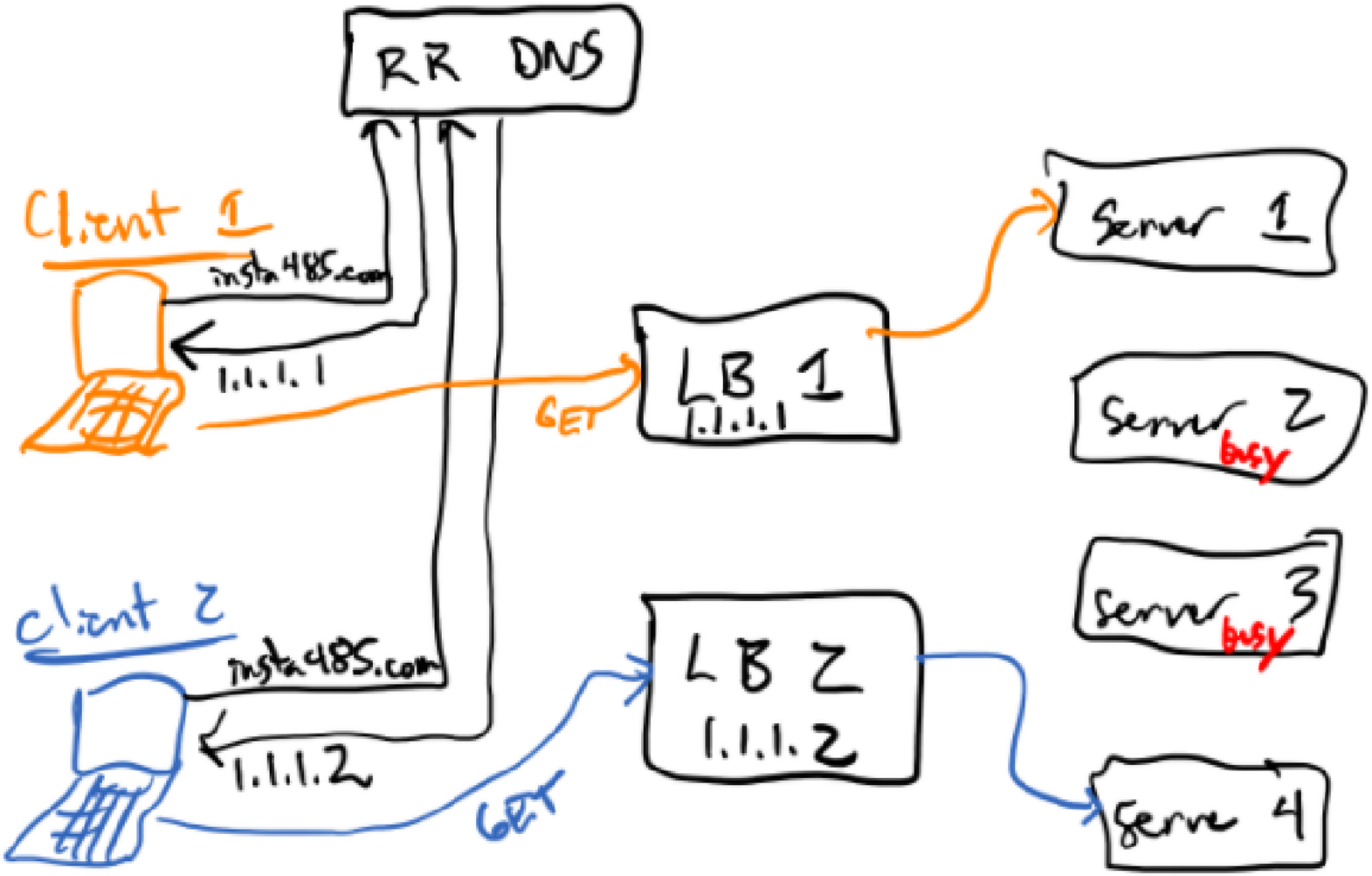
Load balancer description

- Load balancer, AKA proxy server
- "Middleman" forwards requests to backend servers
- End user doesn't need to know about them

Problem with load balancer

- Problem: Load balancer is a single point of failure
- Solution: Round robin DNS to multiple load balancers

Load balancer with Round Robin DNS



Agenda

- Multi-process, multi-thread, and asynchronous
- Round robin DNS and load balancing
- **Hardware virtualization**
- Containerization

Scaling servers

- Many servers running our server-side dynamic pages code
  - Example: Python/Flask via Gunicorn
- Rented from an IaaS provider (Data Center)

Datacenter problems: aggregation

- IaaS provider has many **different computers**
  - Different CPUs, memory, etc.
- Difficult to customize the program for different server configurations
  - E.g. more threads on more powerful computers
- How can we *aggregate* workloads together on machines?

Datacenter problems: isolation

- IaaS provider has many **different customers**
- Customer programs have different requirements
  - Linux kernel versions, installed libraries, etc.
- How to securely *isolate* different programs from each other?

Datacenter problems: migration

- Sometimes many users visit our site, sometimes few
- Sometimes machines fail
- How to add more machines when load is high?
- How to *migrate* running server programs when a machine fails?

Hardware virtualization

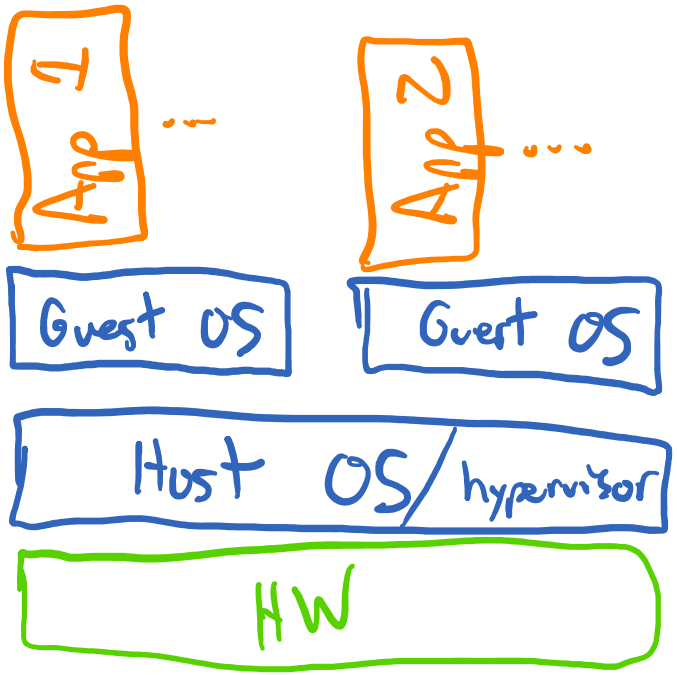
- Problem 1: Workload Aggregration
- Problem 2: Workload Isolation
- Problem 3: Workload Migration
- Solution: Hardware Virtualization

Not hardware virtualization

- Without hardware virtualization, one physical computer runs one operating system

Hardware virtualization

- With hardware virtualization, one physical computer runs multiple operating systems



HW Virtualization vs. virtual environment

- Hardware virtualization and Python virtual environments have nothing to do with each other

- HW virtualization: one physical computer runs multiple operating systems
- Python virtual environment: one operating system runs multiple Python and third-party library versions

Hardware virtualization

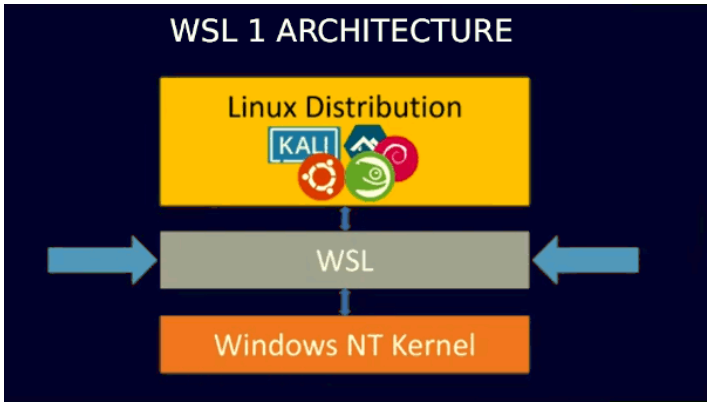
- With hardware virtualization, one physical computer runs multiple operating systems

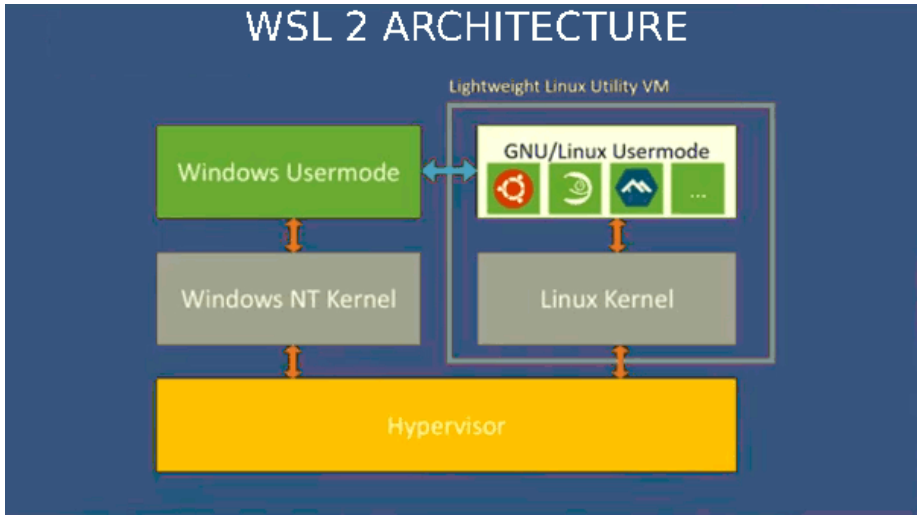
Hardware virtualization terminology

- Host: physical computer running OS and virtualization software
- Guest: operating system(s) being run as “programs”, not on dedicated physical hardware
- Hypervisor: virtualization software runs guests on host
  - VirtualBox, VMWare, et al.
- Hardware emulation
  - Host shares physical resources like network, disk, etc. with guests via virtual hardware

Tangent: WSL2

- Another use for hardware virtualization: Windows Subsystem for Linux 2 (WSL2)
- Run Linux operating system on top of windows
  - Better compatibility with Linux programs





Images from https://fossbytes.com/what-is-windows-subsystem-for-linux-wsl/

Hardware virtualization advantages

- Workload Aggregation
  - Run multiple workloads on each physical machine
  - Run every physical machine at maximum efficiency
- Workload Isolation
  - Each service gets its own preferred OS, libraries, etc.
  - Compromised guest OS won’t affect other guest VMs
- Workload Migration
  - Add more VMs as more users hit your site
  - If there’s a power failure at your East data center, replicate VMs to your Midwest data center

Hardware virtualization disadvantages

- Memory
  - Guest includes a complete OS including drivers, binaries, and libraries
- Slow start
  - 10s ~ 100s to boot a VM

Hardware virtualization disadvantages

- Memory
  - Guest includes a complete OS including drivers, binaries, and libraries
- Slow start
  - 10s ~ 100s to boot a VM

Agenda

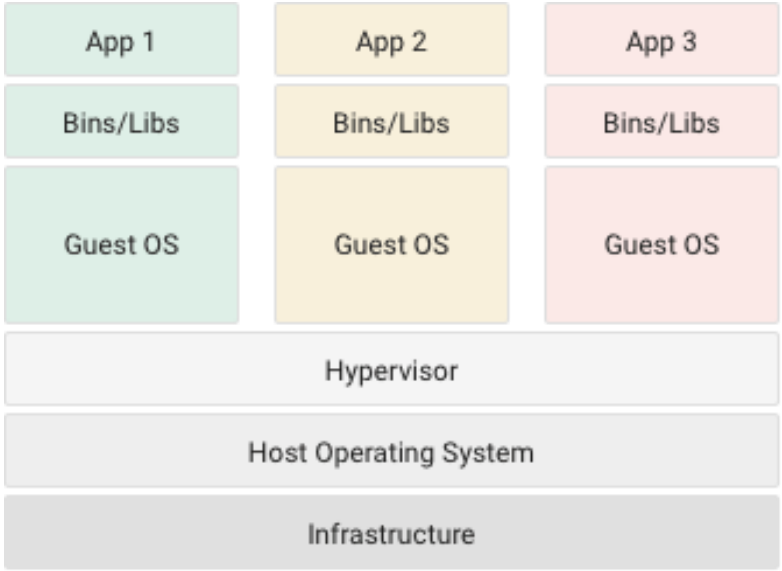
- Multi-process, multi-thread, and asynchronous
- Round robin DNS and load balancing
- Hardware virtualization
- **Containerization**

Containerization

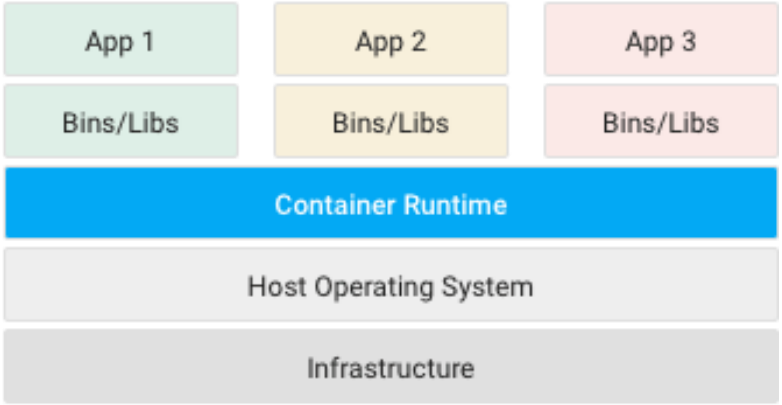
- Problem: Virtualization has high overhead
  - Memory
  - Slow start
- Solution: Containerization
  - Docker is popular
- Key idea: share OS, with its binaries and libraries

Containerization

- Container includes application code and dependencies
- Container does *not* include OS



Virtual Machines



Containers

Example: Virtual machine

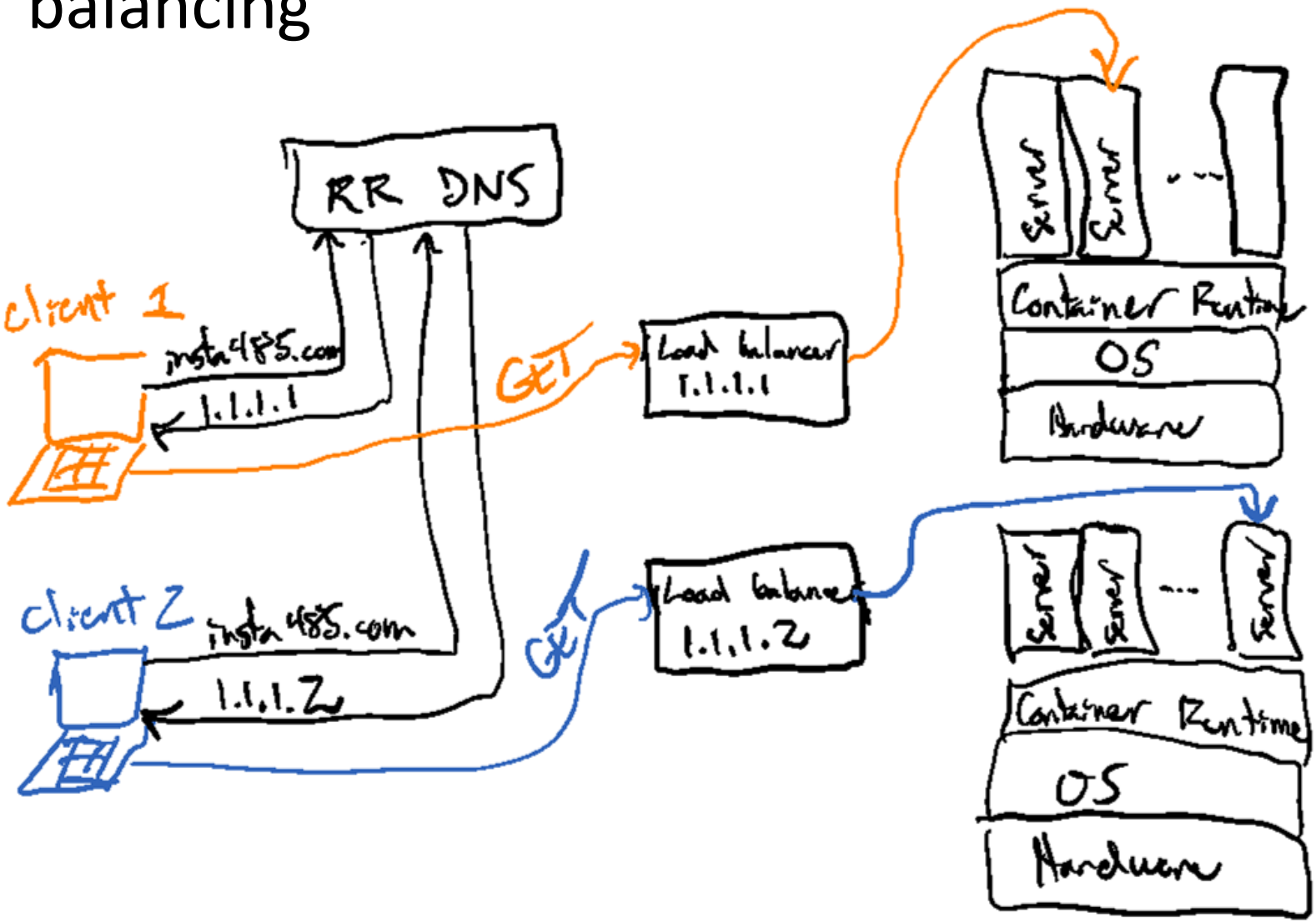
- Install virtualization manager, e.g., VirtualBox
- Create guest virtual machine
  - Install OS on guest VM
  - Install Python and Gunicorn on guest VM
- Copy Flask program `insta485` from host to guest
- SSH into guest
- Run program on guest
  ```
  $ gunicorn insta485:app
  ```
- Stop guest VM

Example: container

- Install container runtime (Docker)
- Create Docker image
  - Copy Flask program `insta485` into container
  - Configure container to execute server
  - `$ docker build`

- Run Docker container from host
- `$ docker run insta485`

Containers, round robin DNS and load balancing



Containerization advantages

- Lower memory footprint
- Faster start-up time 0.1 ~ 1 s

Containerization disadvantages

- All containers that run on a Linux host must be designed to run on Linux
  - Can’t run Windows containers on a Linux host
- A security vulnerability in the OS will impact every container on that host machine

Virtualization vs. containerization

Virtualization

- Run multiple OSs on a single physical machine
- Abstraction of hardware
- Replication
- Isolation
- High mem overhead
- Slow startup
- Stateful

Containerization

- Run multiple apps on a single virtual or physical machine
- Abstraction of OS
- Replication
- Isolation
- Low mem overhead
- Fast startup
- Stateless

Stateful vs. stateless

- Example server-side dynamic pages app that logs to /var/log/insta485.log

```
127.0.0.1 - [06/May/2021 16:50:09] "GET / HTTP/1.1" 200 -
127.0.0.1 - [06/May/2021 16:50:09] "GET /static/js/bundle.js HTTP/1.1" 200 -
127.0.0.1 - [06/May/2021 16:50:09] "GET /api/v1/posts/ HTTP/1.1" 200 -
```

- Run it, make a few requests, restart
- What happens to the log in a virtual machine?
- What happens to the log in a container?

Stateful virtual machine example

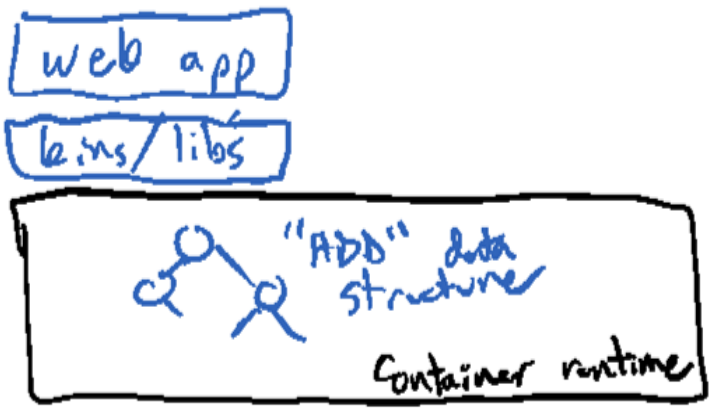
- VMs have a virtual, persistent hard drive
  - Hard drive is a file on the host computer
  - VM writes to the HDD -> file modified
- Restart VM, file is restored, including changes

Stateful virtual machine example

- VMs have a virtual, persistent hard drive
  - Hard drive is a file on the host computer
  - VM writes to the HDD -> file modified
- Restart VM, file is restored, including changes

Stateless container example

- Container software layer emulates hard disk drive
  - Hard drive is a data structure in the container runtime
  - Container writes to the HDD -> data structure modified
- Restart container, *changes are lost*
  - This is a feature! Separation of storage and compute.

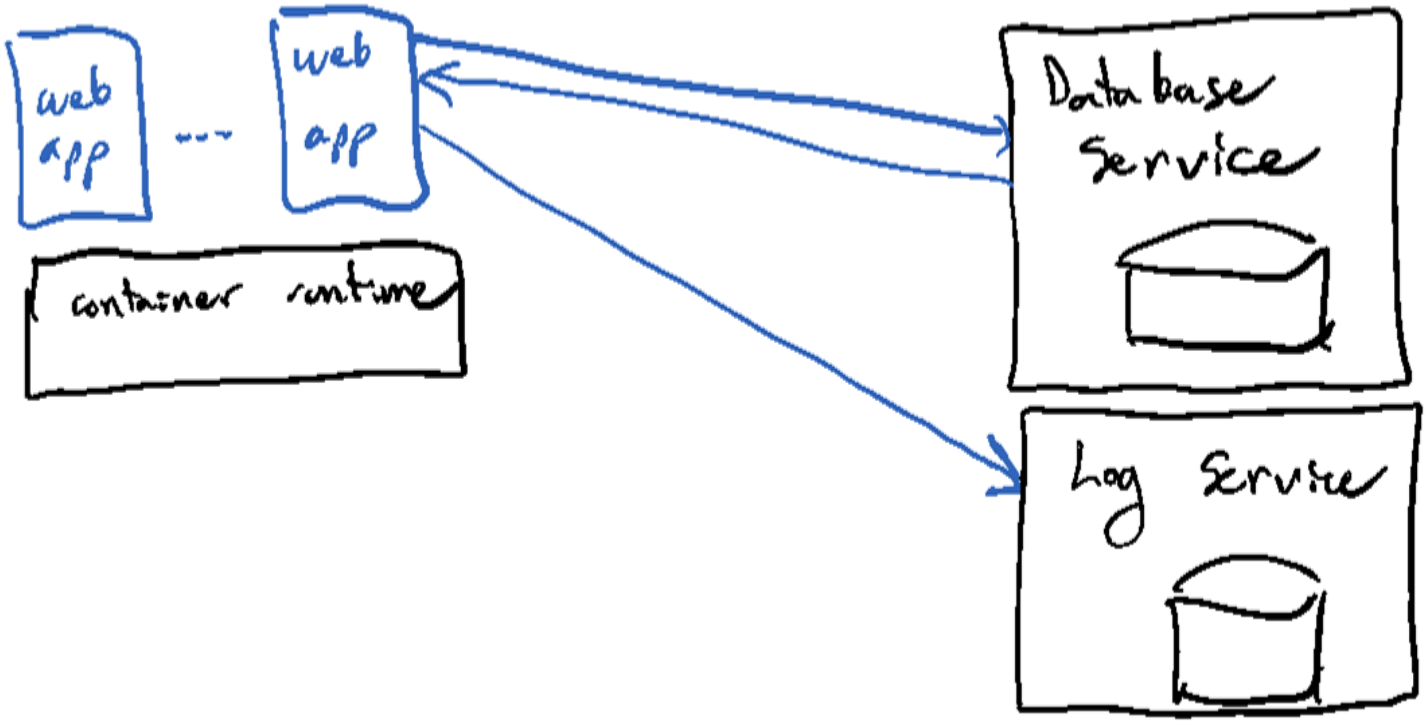


Service-oriented architecture (SOA)

- Service-oriented architecture (SOA)
- **Not SOA:** run entire web app on one machine
- **SOA:** Break up web app into services that communicate over a network
- Run different services on different machines
  - Static pages servers
  - Server-side dynamic pages servers
  - Database servers
  - Log storage servers
  - Etc.

Microservice architecture

- Microservice architecture is a service-oriented architecture with "smaller pieces"
- Common with containerization



Check your understanding

- We talked about scaling server-side dynamic pages today
- How do we scale a client-side dynamic pages application?

Check your understanding

- How do we scale a client-side dynamic pages application?
  - Scale the JavaScript distribution with same techniques as static pages
    - bundle.js via CDN
  - Scale the REST API using same techniques as server-side dynamic pages
    - REST API server code in a container on PaaS

Scaling road map

- Last time: scaling static pages
- Today: scaling dynamic pages
- Next time: scaling storage
  - Database
  - Media uploads